

# Model Context Protocol & GitHub Copilot

---

*Connect AI to your enterprise tools, data sources, and services — through one open standard.*

Open standard

Architecture

GitHub MCP

Registry

Governance

Custom server

Sandeep Soni · Deccansoft Software Services Pvt Ltd

Deck 11 · GH-300 Certified GitHub Copilot Expert Training

# What is MCP — "USB-C for AI"

*One protocol that connects any AI model to any data source or tool. Created by Anthropic in 2024; governed by the Linux Foundation's Agentic AI Foundation since December 2025.*

## Open standard

Vendor-neutral spec; multiple host implementations.

## Universal connectivity

One protocol for databases, APIs, file systems, internal tools.

## Security built-in

Fine-grained permissions; enterprise policies for every connection.

## AI-model agnostic

Works with GPT, Claude, Gemini — any LLM supported by your host.

# Why MCP matters — four pillars for enterprise AI

## Access enterprise systems

Connect AI to databases, APIs, internal tools, legacy systems — without writing a custom integration each time.

## Standardised integration

One protocol replaces dozens of bespoke connectors. Build once, connect everywhere.

## Security & control

Admins control which MCP servers are allowed, with fine-grained policies and trust verification.

## Context enrichment

AI gets real-time data from your systems — eliminating hallucination by grounding responses in facts.

# Without MCP vs with MCP — same question, different outcome

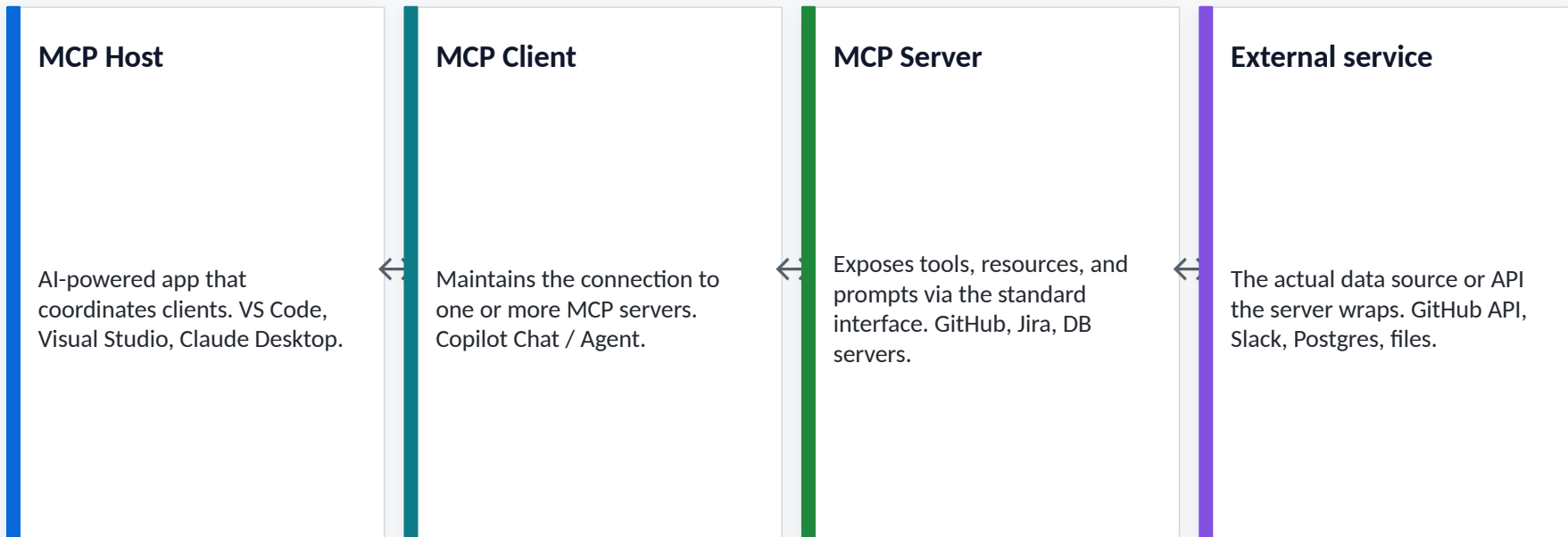
## Without MCP

- ① User → Copilot → GPT:  
*"What should I wear in Paris?"*
- ② GPT has no access to real-time weather data.
- ③ Result: hallucination.  
*"It's warm in Paris" — actually 10°C.*

## With MCP

- ① User → Copilot: "What should I wear in Paris?"
- ② Model sees weather tool available; calls `GetTemperature(Paris)`.
- ③ MCP server returns: 10°C.
- ④ Model + real data = accurate advice: "Wear a warm jacket and layers."

# MCP architecture — four components working together



# Communication flow — from prompt to grounded response



walkthrough

//Example — Jira bug query via MCP

User: "What open bugs are assigned to me?"

Copilot: identifies intent → "needs Jira data".

LLM: selects tool → `get_assigned_issues(user=@me, status=open)`.

MCP: Jira server queries the Jira REST API.

Result: 3 bugs returned with title, priority, sprint.

Output: Copilot renders a formatted list in chat — grounded in real data.

# GitHub MCP Server — the one every developer should connect first

## Capabilities

- Repository management + code browsing.
- Issues and PR automation (create, update, manage).
- Code search across your organisation.
- CI/CD and Actions workflow monitoring.
- Security: Dependabot alerts, code scanning.
- Invoke the Cloud Coding Agent via MCP.

## Key tools

`search_code`

`list_pull_requests`

`create_pull_request`

`get_issue_details`

`get_file_contents`

`create_issue`

`request_copilot_review`

# mcp.json — configure servers; remote HTTP is recommended

mcp.json · remote

```
// .vscode/mcp.json — recommended
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": {
        "Authorization": "Bearer${input:github_pat}"
      }
    }
  },
  "inputs": [{
    "type": "promptString",
    "id": "github_pat",
    "description": "GitHub PAT",
    "password": true
  }]
}
```

## Server types

**Remote (http)** — hosted externally. Recommended for shared servers like GitHub.

**Local (stdio)** — runs on your machine via command. Useful for local DBs and filesystem servers.

## IDE support

VS Code 1.102+ (GA), Visual Studio 17.14+, JetBrains (preview), Eclipse / Xcode (preview).

Commit .vscode/mcp.json to the repo to share the configuration with your team.



# GitHub MCP Registry — discover and install servers

## What it is

- Curated catalog of MCP servers — available via [docs.github.com](https://docs.github.com) and the MCP Registry.
- Partner + community-contributed entries.
- One-click install from the VS Code Extensions view.
- Browse: Ctrl+Shift+X → type @mcp.

## Partner ecosystem

- Figma — Design-to-code via Dev Mode MCP.
- Postman — API platform inside the IDE.
- Terraform — infrastructure management.
- Dynatrace — observability + performance.
- Azure DevOps — work items, pipelines.

## How to install

1

Open Extensions  
(Ctrl+Shift+X)

2

Type @mcp in the search  
bar

3

Browse available servers

4

Click Install → User or  
Workspace

# Enterprise MCP governance — admin controls

## Setting MCP policies

- Enterprise Settings → AI Controls → MCP.
- Enable "MCP servers in Copilot".
- Set MCP Registry URL (BYOR or Azure API Center).
- Choose access policy: Allow All / Registry Only / Block All.

## Key notes

- Policy applies to Business + Enterprise plans.
- Free/Pro/Pro+ are not governed by org policy.
- Azure API Center provides a managed registry.
- Coding Agent MCP: configure in repo settings.

## Coding Agent + MCP

Configure MCP servers for the Cloud Coding Agent in Repository Settings → Copilot → Coding Agent → MCP Configuration. Supports Azure DevOps MCP, Sentry, custom servers. The agent uses tools autonomously without per-action approval — pick allowlists carefully.

# MCP security — trust, secrets, and least privilege

## Trust verification

Only add servers from trusted sources. VS Code prompts before starting any new server — read the publisher.

## Secrets management

Never hardcode API keys. Use input variables / env files. Coding Agent secrets need the `COPILOT_MCP_` prefix.

## Least-privilege tools

Enable only the tools you need. Disable unused servers. Use the Tools picker to toggle per-session.

## Enterprise policies

"Registry Only" mode for strict environments. Allowlists at enterprise / org level. Azure API Center for managed governance.

## Tool-limit awareness

Max 128 tools per chat request (model constraint). Use BYOR / dynamic toolset discovery to soften the cap.

## Audit + review

Cloud-Agent actions show up in PR commits. Local MCP output visible in IDE logs. Monitor for unusual tool patterns.

*Local MCP servers can run arbitrary code on your machine — review before trusting.*

# Real-world scenarios — multi-server MCP workflows

## DevOps — deploy to staging

Shell + GitHub + REST API.  
"Deploy latest main to staging" →  
build → tag → trigger webhook.

## QA — regression suite

Shell + Database + Filesystem.  
"Run full regression and report" →  
execute → query history → write report.html.

## PM — sprint status

GitHub + Jira + Docs.  
"What's blocking Sprint 4?" →  
open PRs + blocked items + decisions.

## Security — audit endpoints

GitHub + DB + Shell.  
"Audit auth endpoints" →  
scan auth code + permission tables + security lint.

# Coding Agent MCP — give the cloud agent its own toolset

`coding-agent-mcp.json`

```
# Repo Settings → Copilot → Coding Agent → MCP
```

```
{
  "servers": {
    "sentry": {
      "type": "http",
      "url": "https://mcp.sentry.io/",
      "headers": {
        "Authorization":
          "Bearer $COPILOT_MCP_SENTRY_TOKEN"
      }
    },
    "azdevops": {
      "type": "http",
      "url": "https://dev.azure.com /.../mcp"
    }
  }
}
```

## What this unlocks

- Assign Copilot an issue that says "fix the top Sentry error of the last hour".
- Agent reads Sentry, locates the offending code, patches it, opens a PR.
- Same loop with Azure DevOps work items, Jira tickets, or your own internal API.

## Watch for

- Secrets must use the COPILOT\_MCP\_ prefix.
- Cloud agent uses tools without per-call approval — keep allowlists minimal.
- PR diffs surface every MCP-driven change for review.

# Build a tiny custom MCP server — demystify the protocol

server.ts

```
//server.ts — Node TypeScript, ~15 lines
import { MCPServer } from '@modelcontextprotocol/sdk/server/mcp.js';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
import { z } from 'zod';

const server = new MCPServer({ name: 'hello-mcp', version: '0.1.0' });

server.tool('say_hello', { name: z.string() }, async ({ name }) => ({
  content: [{ type: 'text', text: `Hello, ${name}!` }]
}));

await server.connect(new StdioServerTransport());
```

## Wire it up

- ① `npm i @modelcontextprotocol/sdk zod`
- ② Add to `mcp.json`:  

```
{ type: "stdio",
  command: "node",
  args: ["server.ts"] }
```
- ③ Restart Chat. Toggle the tool on. Try:  
*"Use say\_hello with name=Sandeep"*

# Key takeaways

1

## MCP = USB-C for AI

One open standard; any model, any tool. Linux Foundation AAIF stewardship.

2

## Four components

Host → Client → Server → External service. Same shape for every integration.

3

## Grounding beats hallucination

With MCP, the model answers from real data, not training-data lore.

4

## Remote HTTP for shared servers

mcp.json with type:http; commit it to share with the team.

5

## Govern with allowlists

BYOR registry / Registry Only. Coding Agent secrets prefixed COPILOT\_MCP\_.

6

## Build your own when ready

~15 lines of TS + an SDK. The protocol stops being magic the moment you write a server.

# Thank you.

*Connect one MCP server today. Watch what changes in your next prompt.*

---

Sandeep Soni · Deccansoft Software Services Pvt Ltd

*GH-300 · Model Context Protocol · Deck 11*